INTRODUCCIÓN A R Y RSTUDIO

R es un lenguaje de programación para el análisis estadístico y visualización de datos. Frente a otras alternativas que ofrece el mercado, R tiene algunas ventajas destacables

• R es un lenguaje libre y de código abierto. A diferencia de otros programas. Además, al ser un software de código abierto podemos acceder al código desarrollado en cada una de sus funciones y librerías. Esto permite que podamos reutilizar el código desarrollado por otros programadores para recodificarlo y añadirle nuevas funcionalidades.

• R es actualizado a diario por cientos de programadores alrededor del mundo, lo que hace que cualquier análisis o técnica estadística pueda estar disponible.

• R permite la importación y exportación de datos en un gran número de formatos. Puede trabajarse con datos en texto plano (extensiones .txt o .csv), pero también con formatos de diferentes softwares (.xls por ejemplo).

• Es un lenguaje multiplataforma, lo que permite ejecutar sus scripts en Windows, Mac, Linux, etc.

INSTALACIÓN DE R Y RStudio

Para instalar R en su equipo navegue hasta el siguiente enlace e instala la versión que se corresponda con el sistema operativo de su equipo. En el enlace podrá encontrarse el archivo de instalación para el sistema operativo correspondiente:

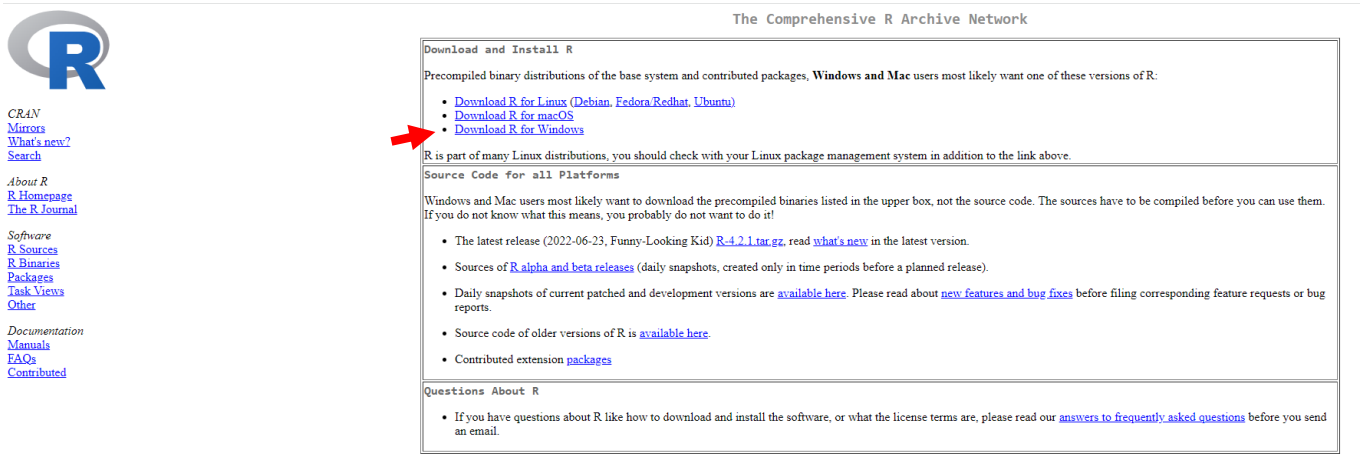
https://cran.r-project.org

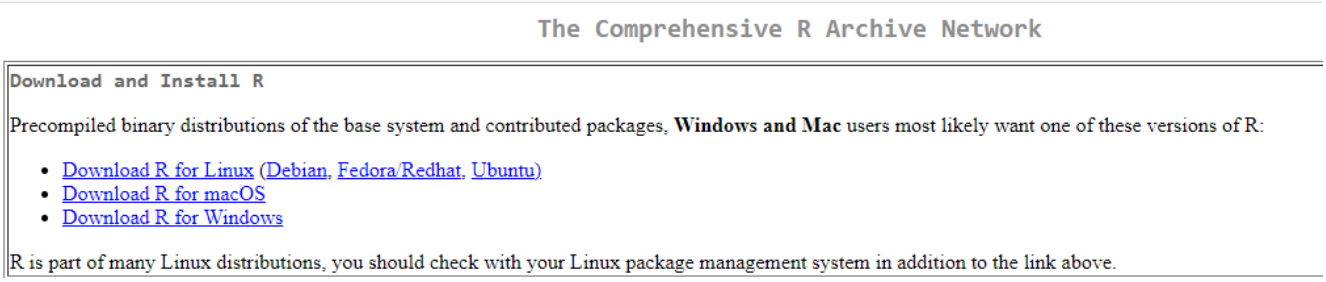
Una vez comprobado que R ya se encuentra en el equipo, el siguiente paso consiste en instalar RStudio. Este programa es un entorno de desarrollo integrado (IDE) para el lenguaje de programación R, lo que facilita las tareas de desarrollo y programación de código, que puede descargarse a través del siguiente enlace:

https://rstudio.com/products/rstudio/download/

INSTALAR LENGUAJE R

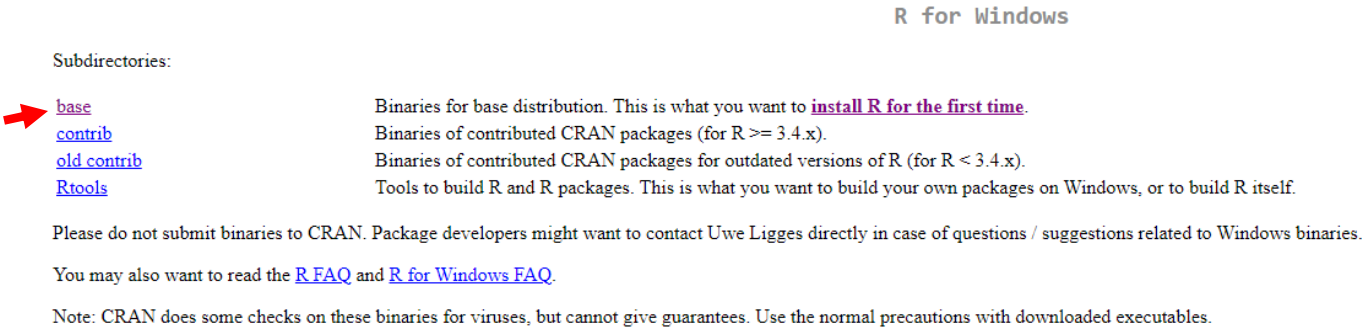
Se debe ingresar a la pagina: https://cran.r-project.org/, y seleccionar el sistema operativo





Se recomienda instalar la versión más frecuente

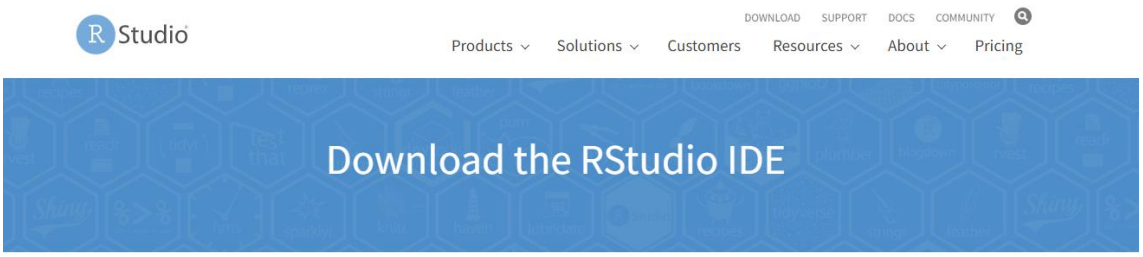
Ingresar a base

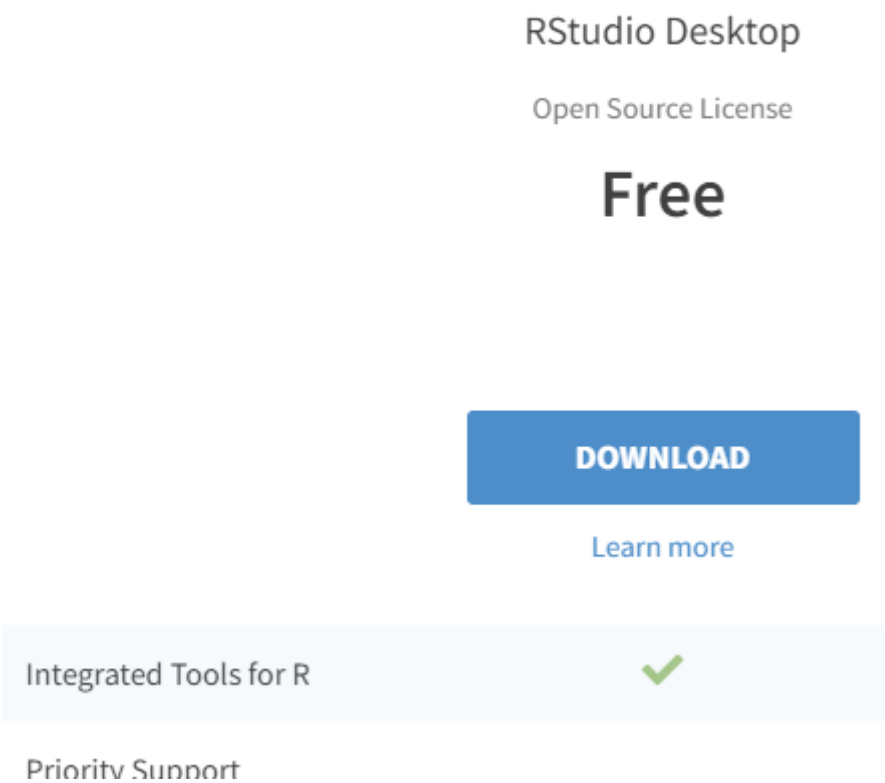


Luego descargar

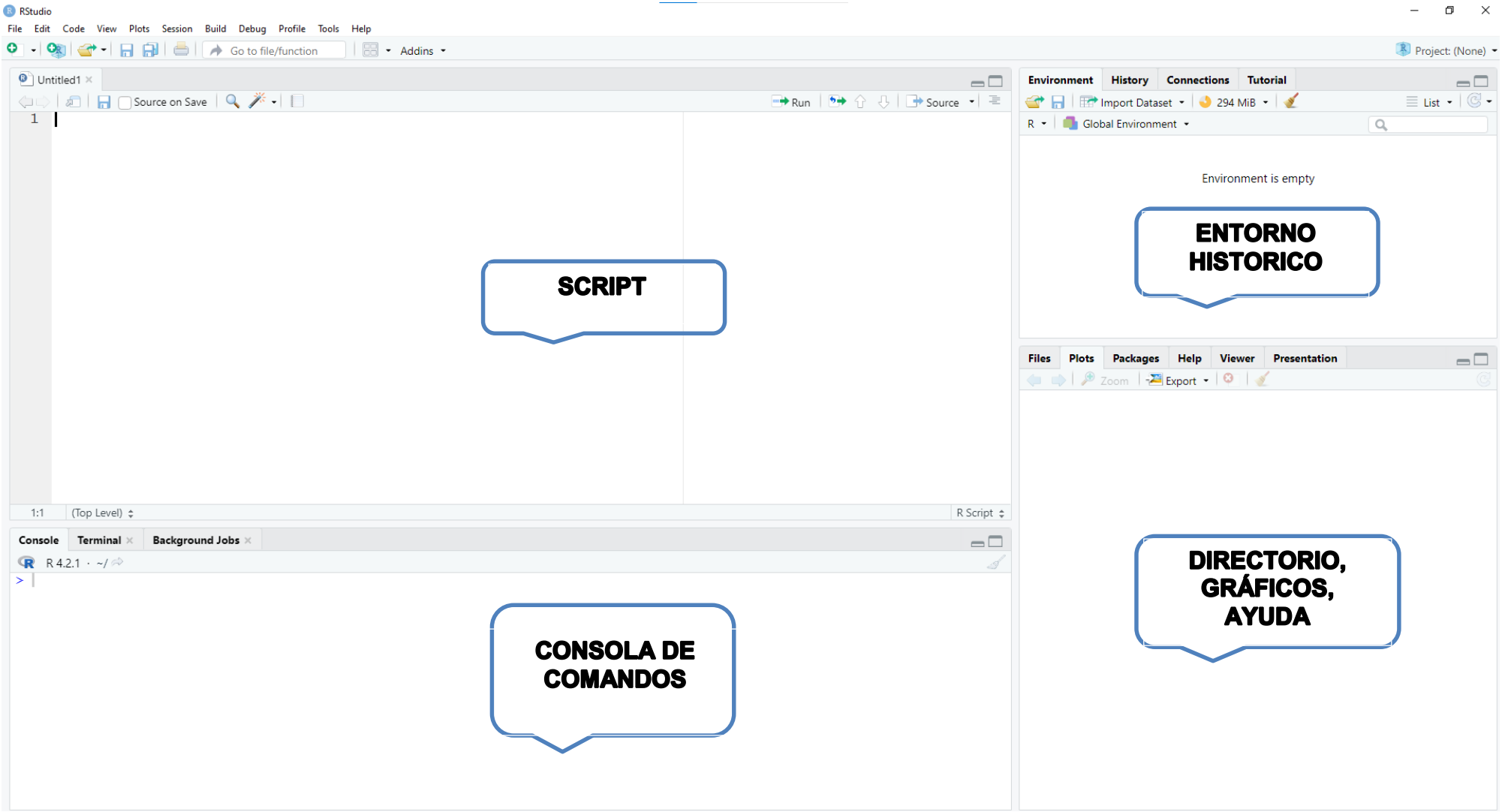
INSTALAR RStudio

Se debe ingresar a la pagina: https://rstudio.com/products/rstudio/download/





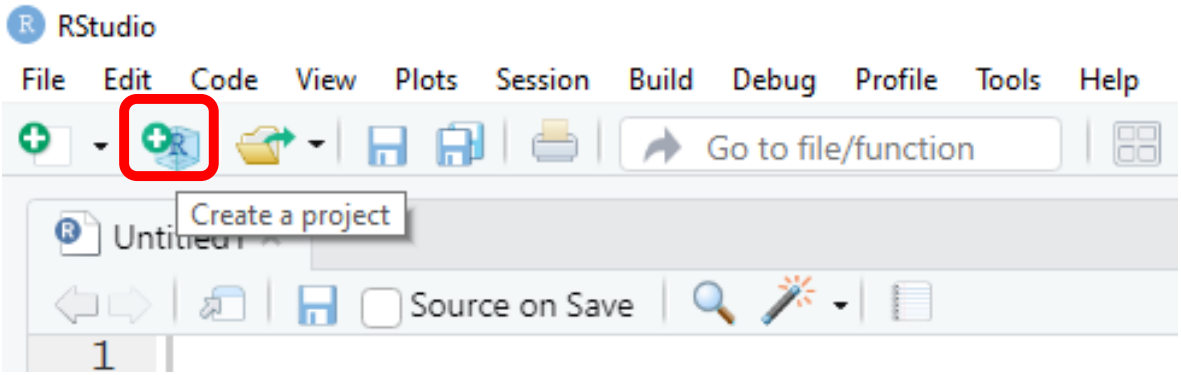
RStudio cuenta con diversos paneles o subventanas donde se combinan todos los elementos necesarios para llevar a cabo cualquier desarrollo en R


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Addins
Project: (None)
Untitled1
Source on Save
Run
Source
1:1
(Top Level)
R Script
SCRIPT
Environment History Connections Tutorial
Import Dataset
294 MiB
List
R
Global Environment
Environment is empty
ENTORNO HISTORICO
Files Plots Packages Help Viewer Presentation
Zoom
Export
DIRECTORIO, GRÁFICOS, AYUDA
Console Terminal Background Jobs
R 4.2.1 · ~/
CONSOLA DE COMANDOS

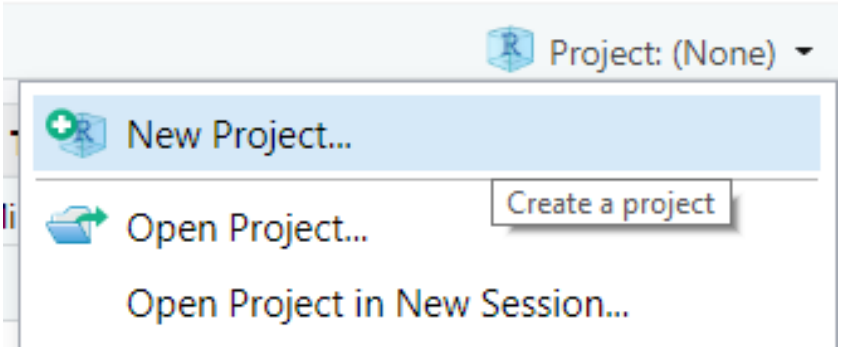
CREAR PROYECTO EN RSTUDIO

Para crear un proyecto en RStudio lo podemos hacer de dos formas

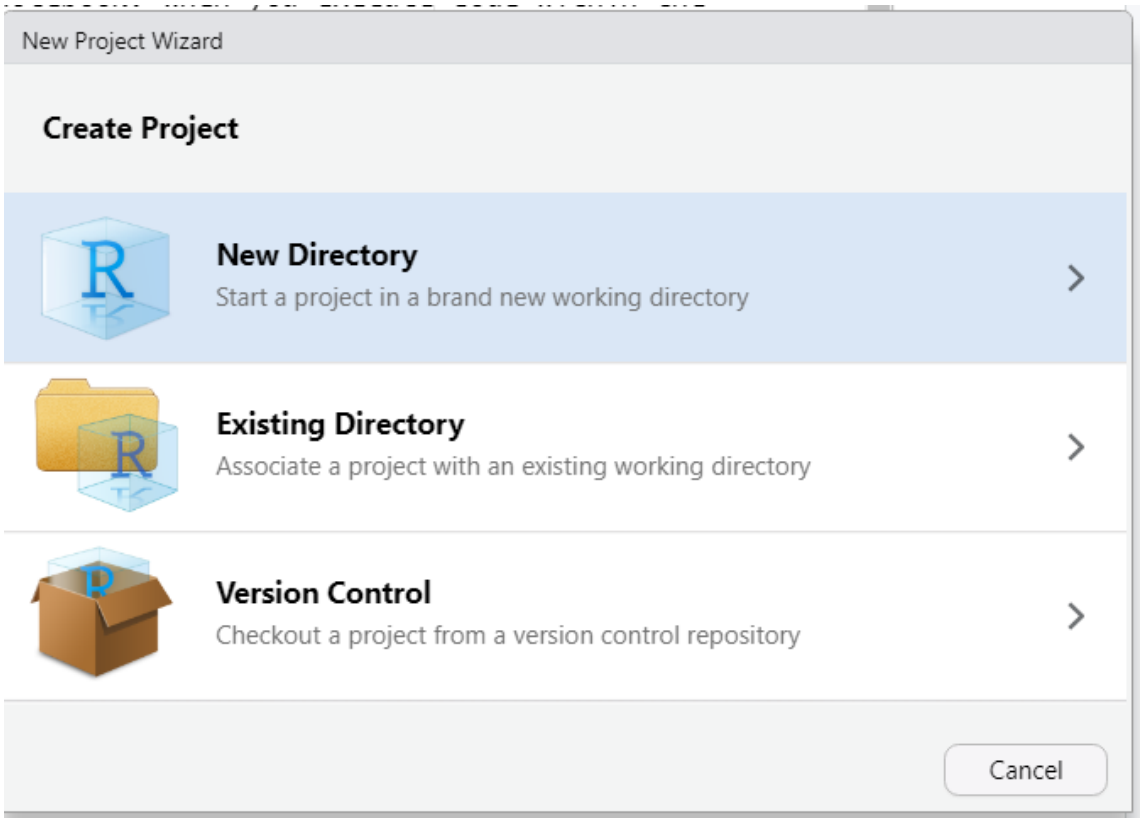
1. Ir a la parte superior izquierda y dar click en el siguiente icono



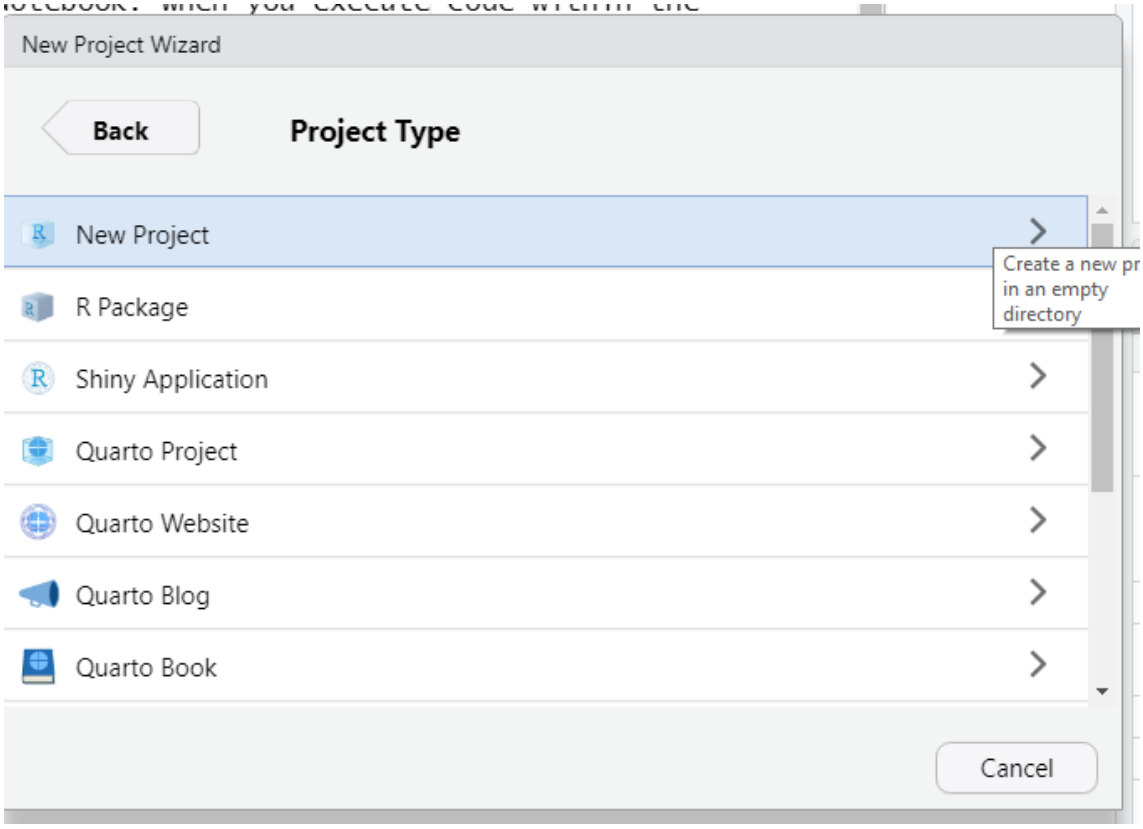
2.- La otra opción es ir a la parte superior derecha y desplegar la ventana de *Project* e ir a *New Project*



Con cualquiera de las dos opciones obtendremos la siguiente ventana. Ingresaremos a *New Directory*

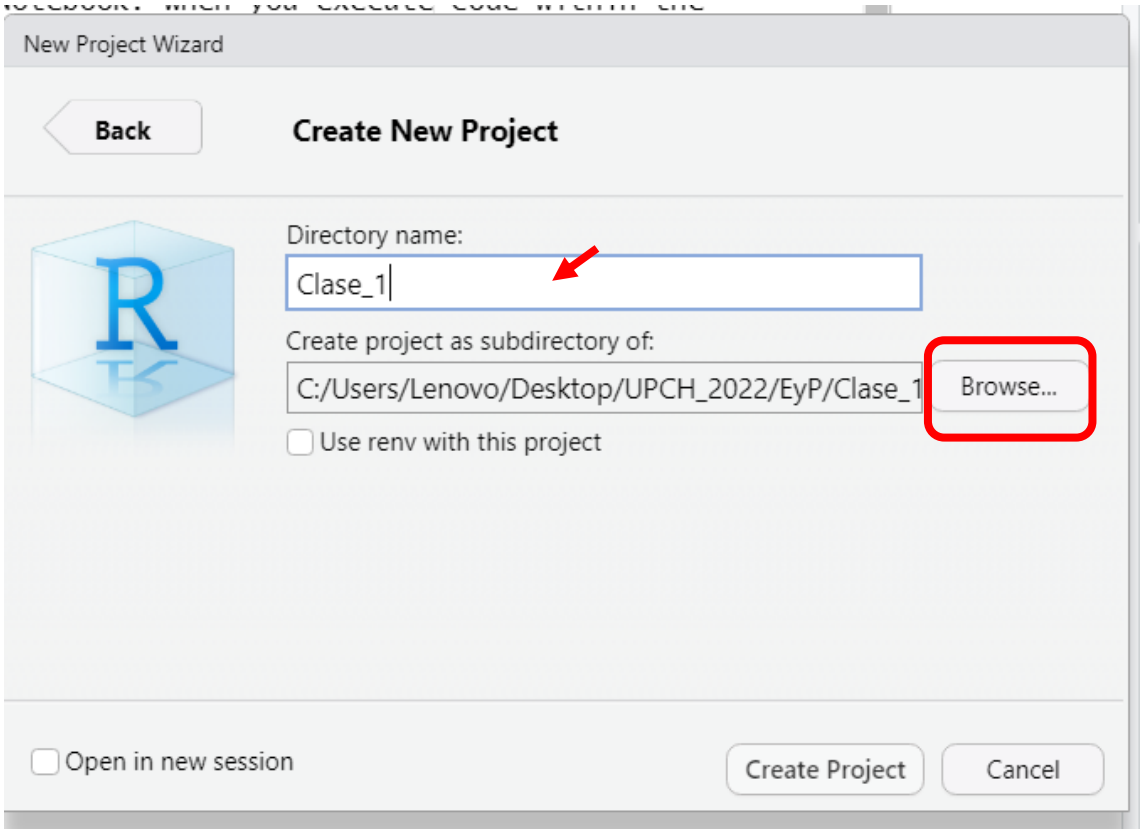


Luego a *New Project*

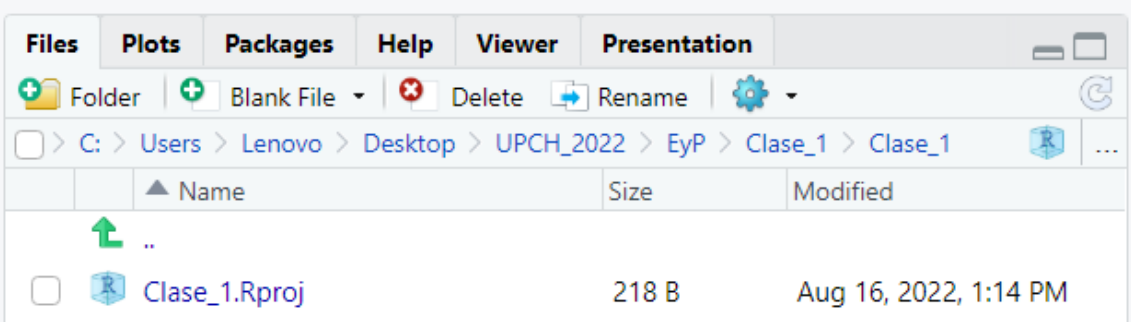


En la siguiente ventana que obtenemos, debemos colocar un nombre para el proyecto (el de tu elección)

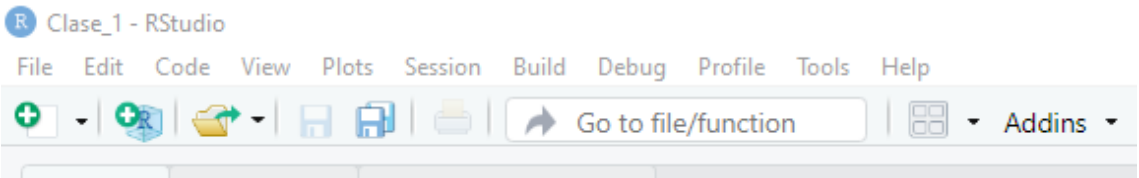
En Browse, debemos ubicar nuestro proyecto



Nos percatamos que en la parte inferior derecha, aparece el nuevo proyecto, como se muestra en imagen

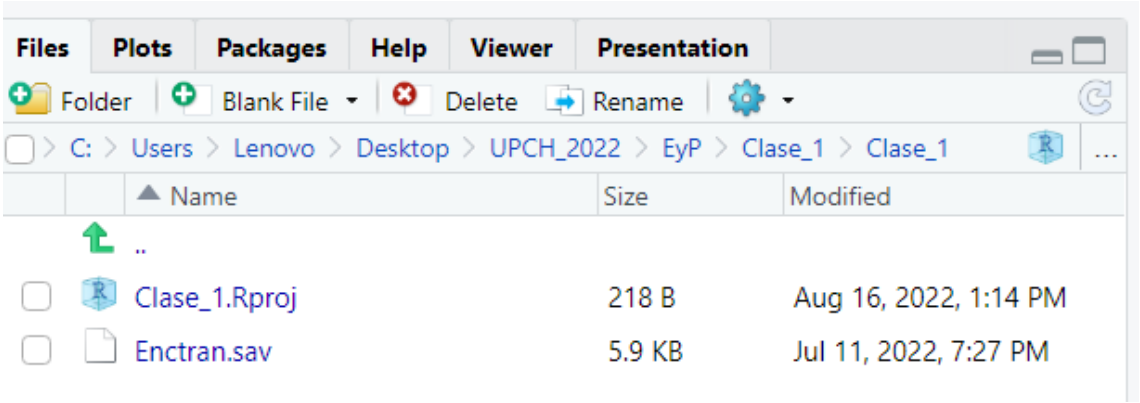


Además en la parte superior de la ventana de RStudio , podemos identificar que estamos trabajando con este proyecto pues aparece el nombre en la parte superior



Ahora en la carpeta donde has ubicado tu proyecto pues colocar los archivos que emplearias para tu trabajo.

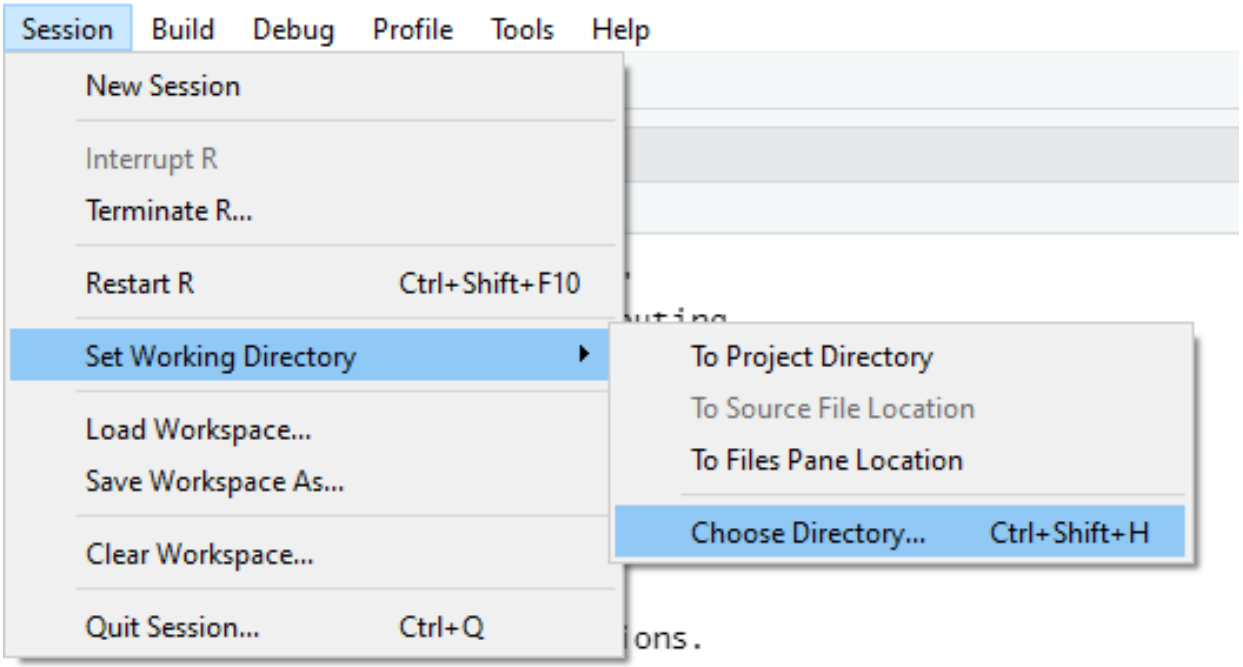
Solo como ejemplo aquí hemos incorporado una base de datos en la carpeta que se genero cuando se creo el proyecto. Luego cuando volvemos al RStudio, también lo podemos visualizar en la carpeta del proyecto.



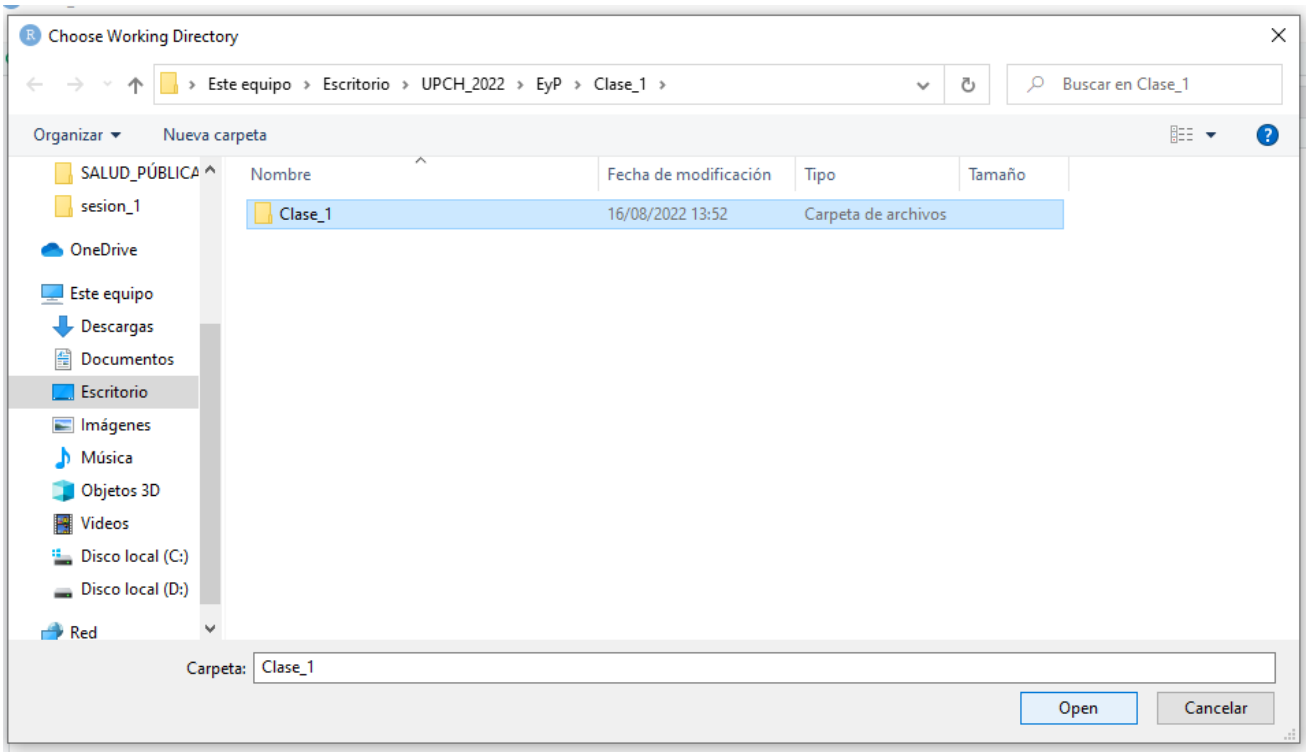
El tener nuestro archivos ubicar en la carpeta del proyecto que estamos trabajando (podemos generar un proyecto por cada sesión de práctica). Nos permitirá trabajar de forma ordenada.

UBICAR DIRECTORIO DE TRABAJO

Para ubicar un directorio de trabajo podemos ir a *Session/Set Working Directory/Choose Directory*



Al ingresar obtendremos la siguiente ventana. Aquí se seleccionara cual será la carpeta de trabajo (previamente debemos haber elaborado la carpeta), y abrimos



En la consola nos saldrá la siguiente función
setwd("ubicación de carpeta, esto variará según la ubicación de la carpeta")

```
Console   Terminal ×   Background Jobs ×
R 4.2.1 · C:/Users/Lenovo/Desktop/UPCH_2022/EyP/Clase_1/Clase_1/
> setwd("C:/Users/Lenovo/Desktop/UPCH_2022/EyP/Clase_1/Clase_1")
>
```

Si deseamos tener información de cual es la carpeta de trabajo, podemos emplear la función ***getwd( )***

```
> getwd()
[1] "C:/Users/Lenovo/Desktop/UPCH_2022/EyP/Clase_1/Clase_1"
>
```

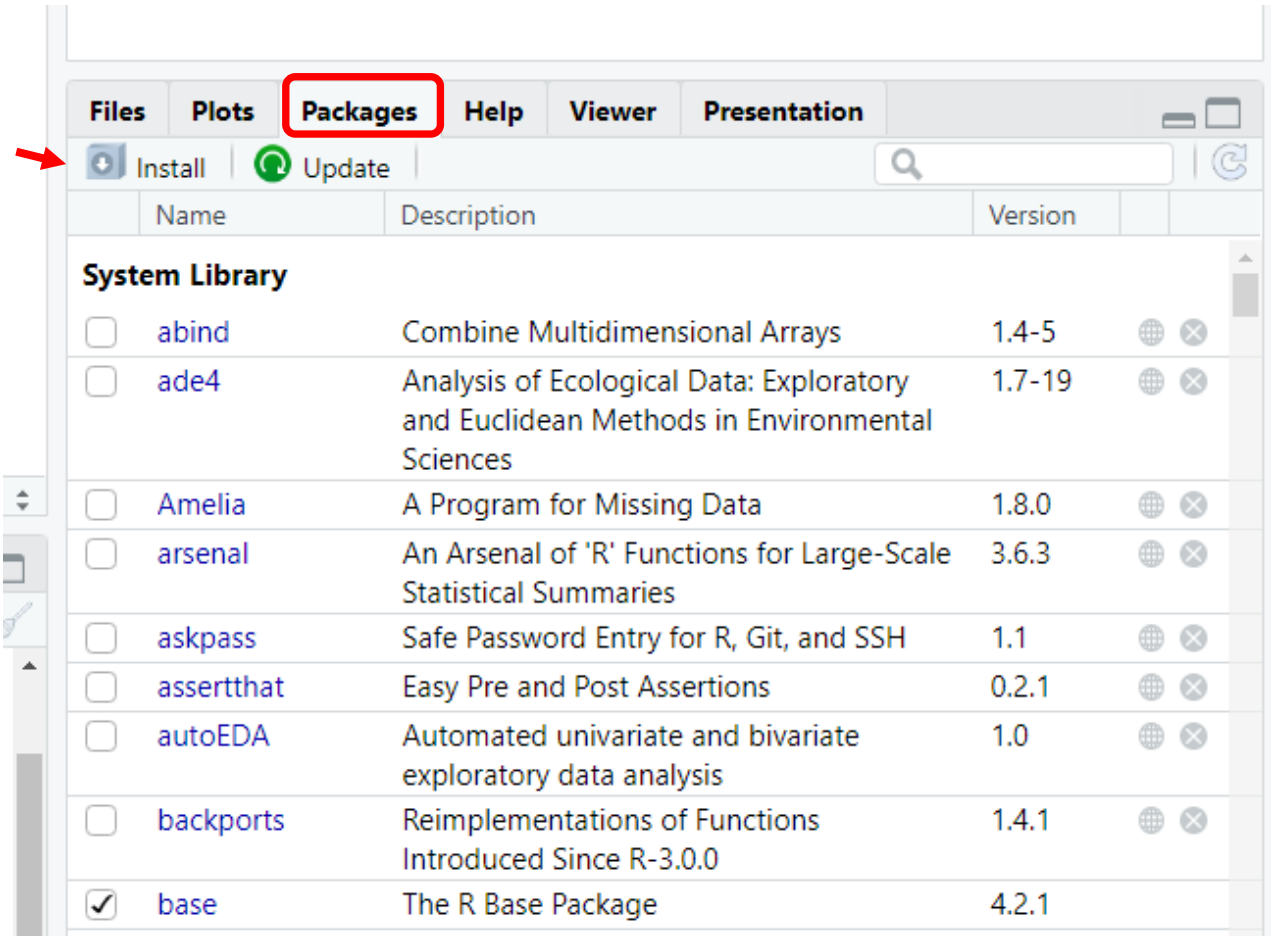
Apreciamos que obtenemos la información en la consola

INSTALACIÓN DE PAQUETES EN RSTUDIO

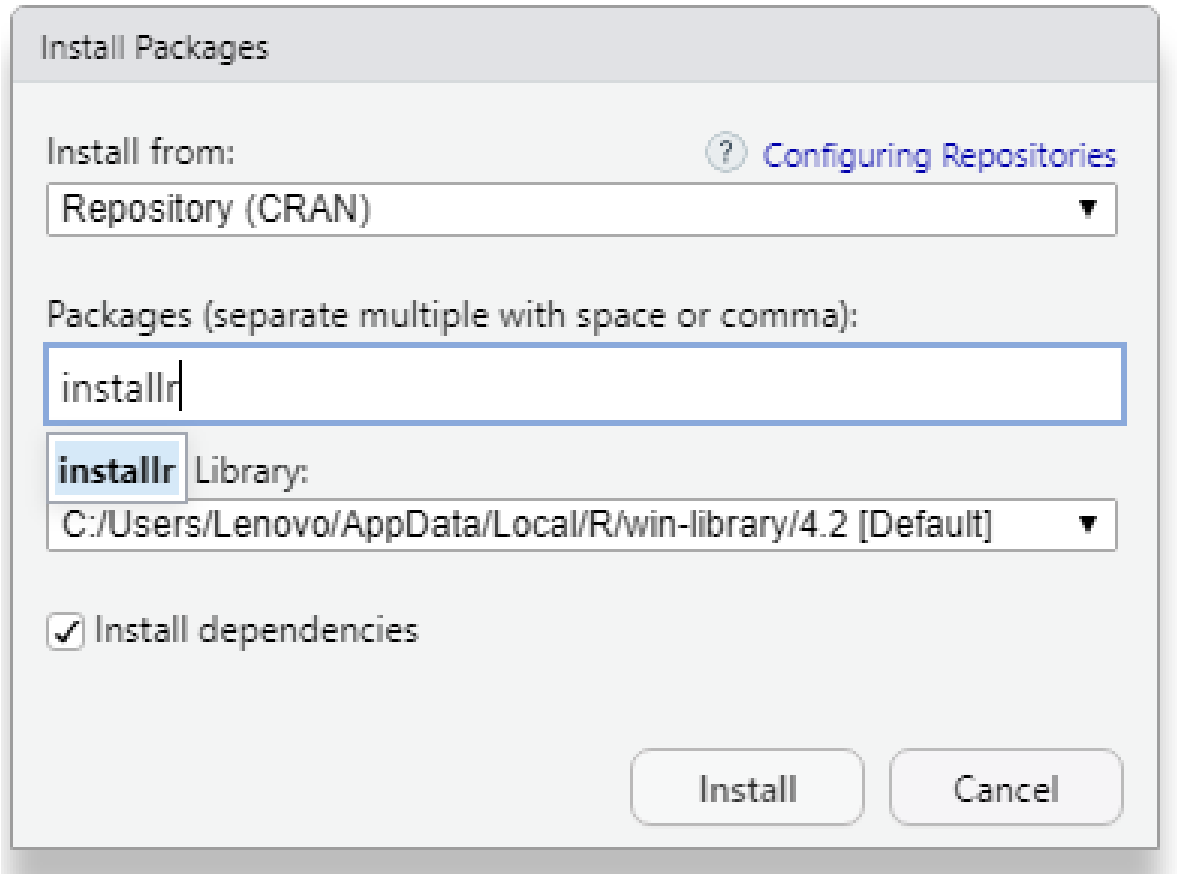
Los paquetes en R son colecciones de funciones y conjunto de datos desarrollados por la comunidad. Estos incrementan la potencialidad de R mejorando las funcionalidades base en R, o añadiendo nuevas.

Frecuentemente se necesita integrar nuevas librerías de R para ayudarnos con el manejo de datos. Estas se encuentran publicadas en "Comprehensive R Archive Network (CRAN)", pero en Rstudio podemos instalarlas directamente siguiendo los siguientes pasos.

En el panel inferior derecho de la pantalla de Rstudio se encuentra la pestaña Packages se da clic en esa pestaña y luego en Install y aparecerá una ventana donde podemos enlistar las librerías que necesitamos



En este caso instalaremos el paquete installr, como se muestra en la imagen



Terminamos en install.

En la consola ,aparecerá la siguiente imagen. Es decir, podríamos haber tipeado en la consola ***install.packages("installr")***

```
> install.packages("installr")
WARNING: Rtools is required to build R packages but is not currently installed. 
g:

https://cran.rstudio.com/bin/windows/Rtools/
Installing package into 'C:/Users/Lenovo/AppData/Local/R/win-library/4.2'
(as 'lib' is unspecified)
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.2/installr_0.23.2.zip
Content type 'application/zip' length 349642 bytes (341 KB)
downloaded 341 KB

package 'installr' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\Lenovo\AppData\Local\Temp\RtmpmyMIkZ\downloaded_packages
```

Lo que nos indica que se logro instalar el paquete

Una vez tenemos instalado un paquete, estamos en disposición de hacer uso de sus funciones.

La forma mas eficiente de trabajar es cargándolo en memoria. Esto lo conseguiremos mediante el uso de la función *library()*

En este caso tipeamos en la consola ***library(installr)***

En la consola aparecerá la siguiente imagen

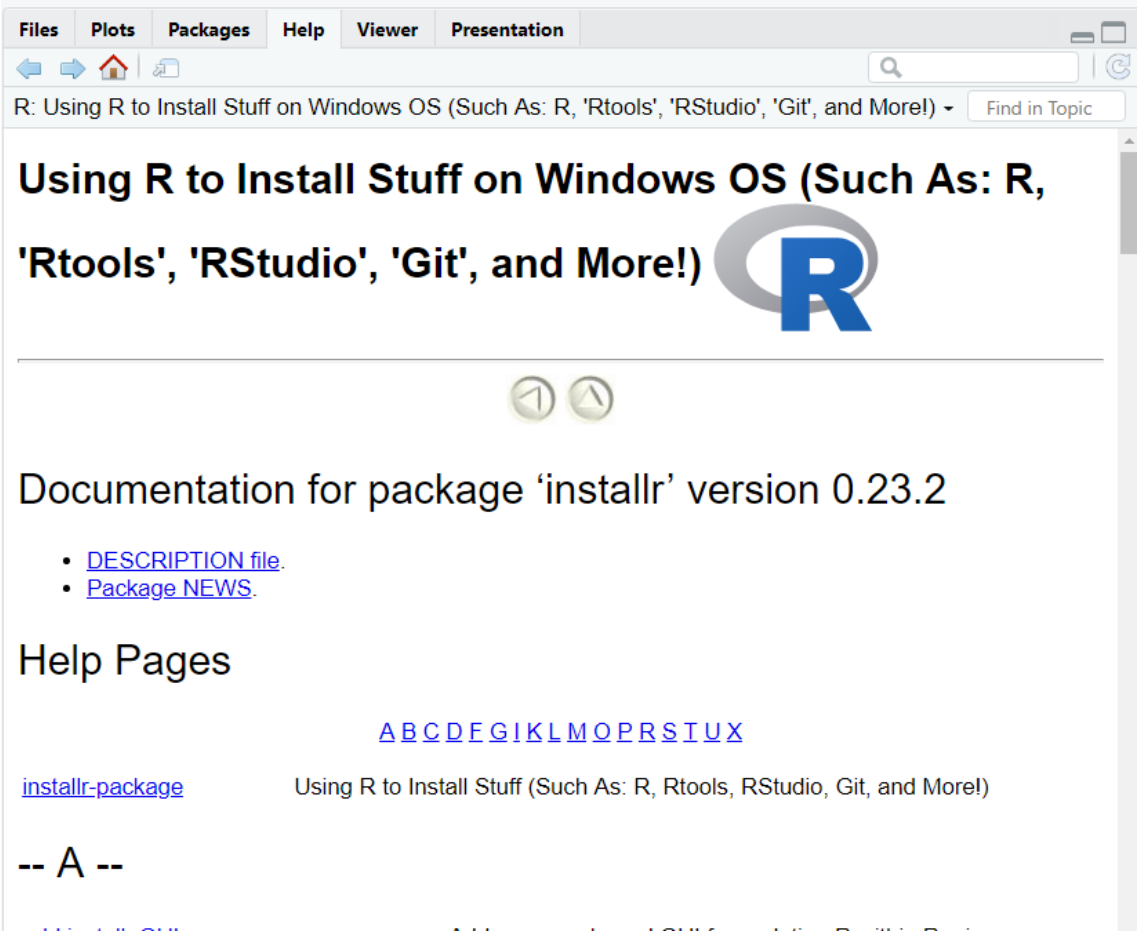
```
> library(installr)

Welcome to installr version 0.23.2

More information is available on the installr project website:
https://github.com/talgalili/installr/

Contact: <tal.galili@gmail.com>
Suggestions and bug-reports can be submitted at: https://github.com/talgalili/installr/issues

                        To suppress this message use:
                        suppressPackageStartupMessages(library(installr))
```

NOTA: Que el argumento de la función *install.packages()* es un vector de tipo carácter y requiere de los nombres de los paquetes entrecomillados, mientras que la función *library()* acepta tanto un vector de tipo carácter como el nombre sin comillas.

RECUERDA que para acceder a las funciones y conjuntos de datos que contiene un paquete, podemos hacerlo mediante: *help(package = "nombre_del_paquete")*

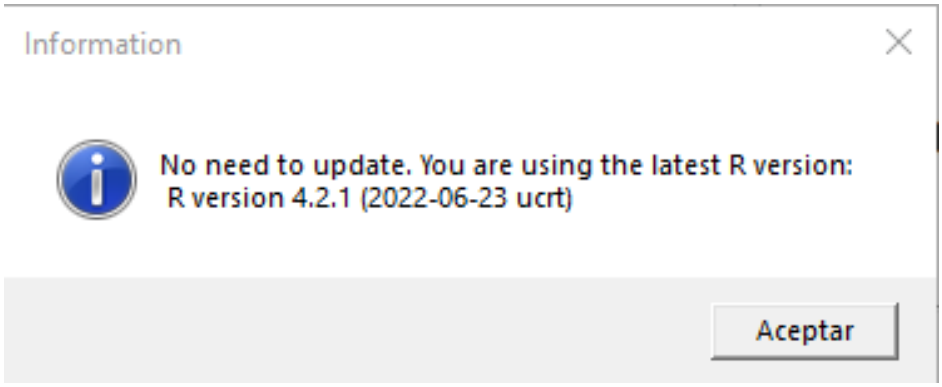
En este caso por ejemplo tipeamos en la consola ***help (package=installr)***

En la ventana inferior derecha se abrirá en la pestaña HELP la información del paquete solicitado

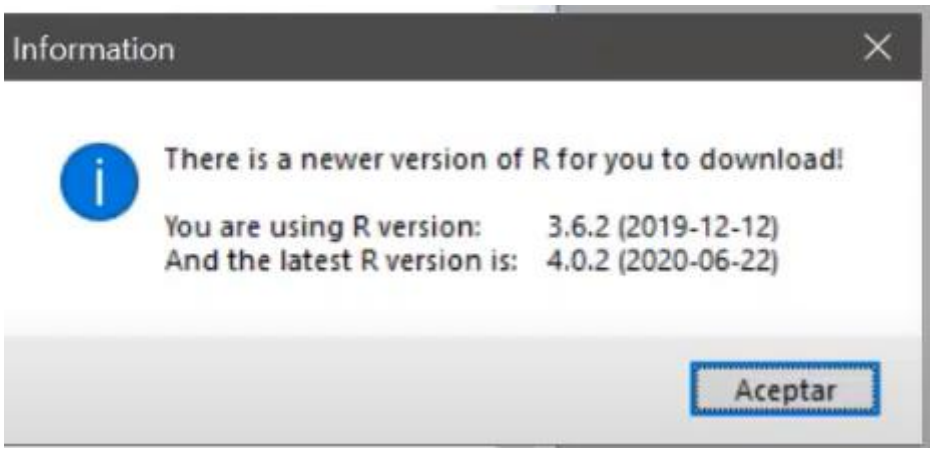


Este paquete tiene la función *updateR(),* que permite actualizar la versión de R. Entonces en la consola tipeamos ***updateR().***

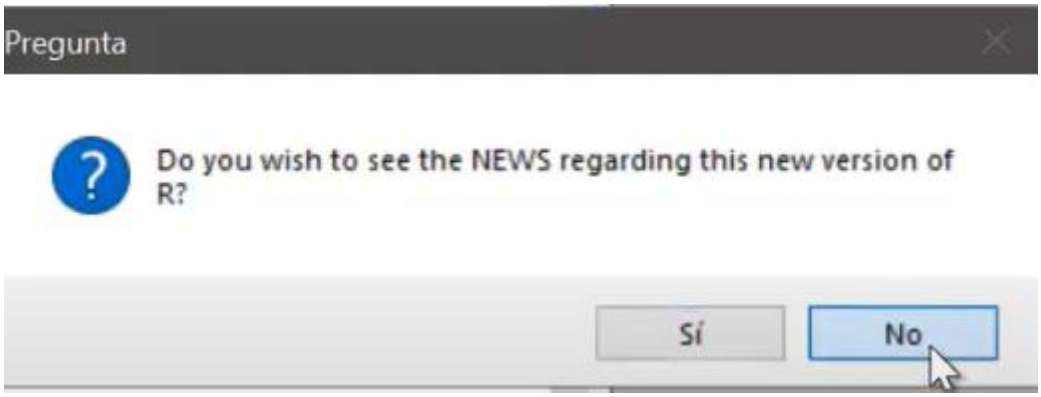
Si cuentas con la última versión te saldrá un mensaje indicando ello:



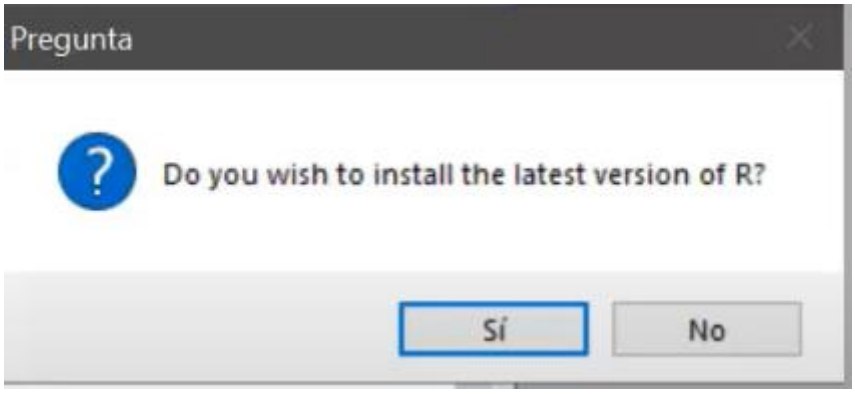
En caso la versión que emplees requiere actualizar saldrá un mensaje similar al de abajo (es un ejemplo cambiará según la versión que tengas y la ultima versión disponible), le das en aceptar.



Luego nos pregunta si deseamos ver las nuevas actualizaciones, en este caso colocaremos no, pues no deseamo visualizar



Posteriormente con pregunta si deseamos actualizar a la última versión

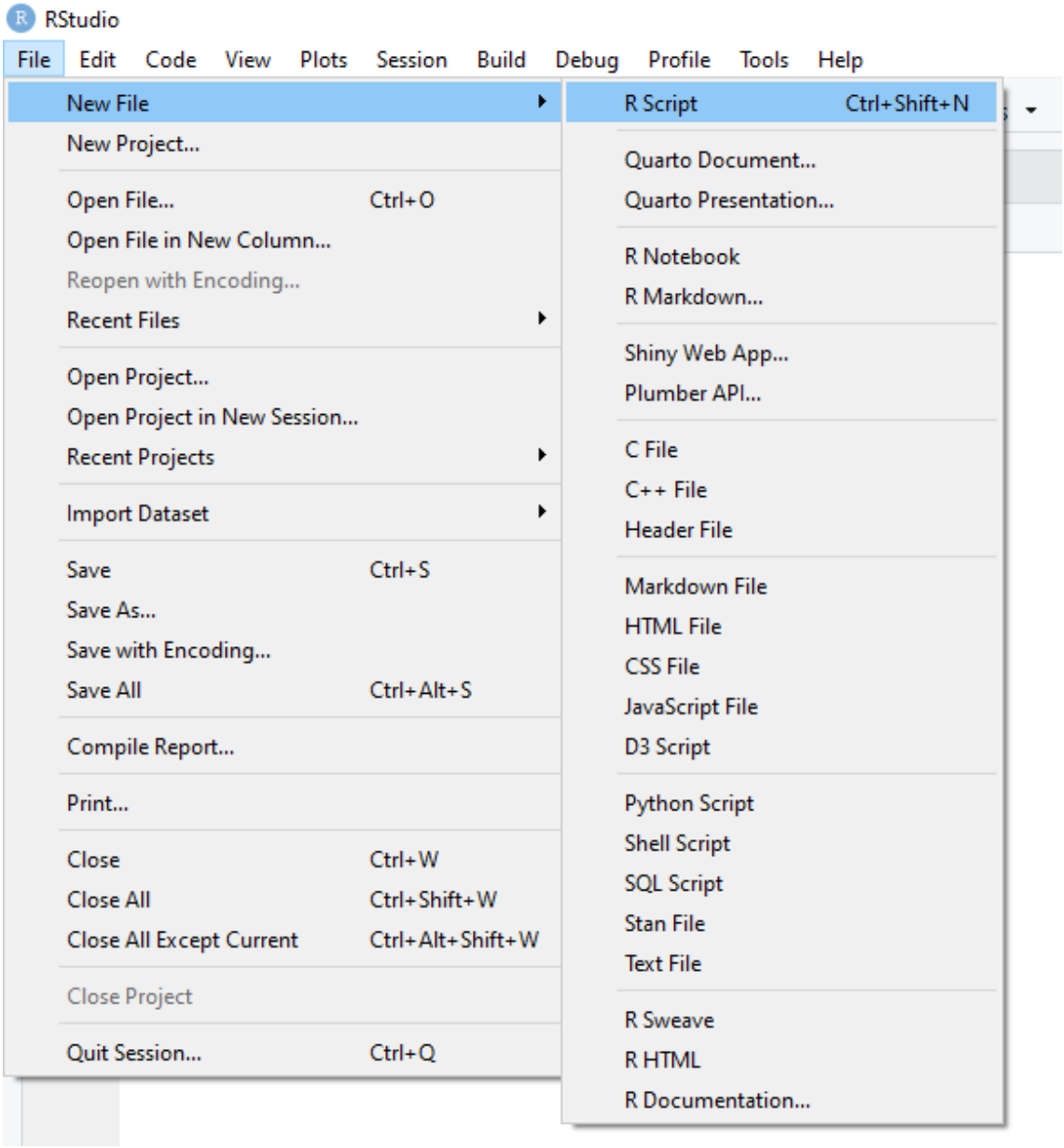


Posterior a ello continua el proceso de instalación

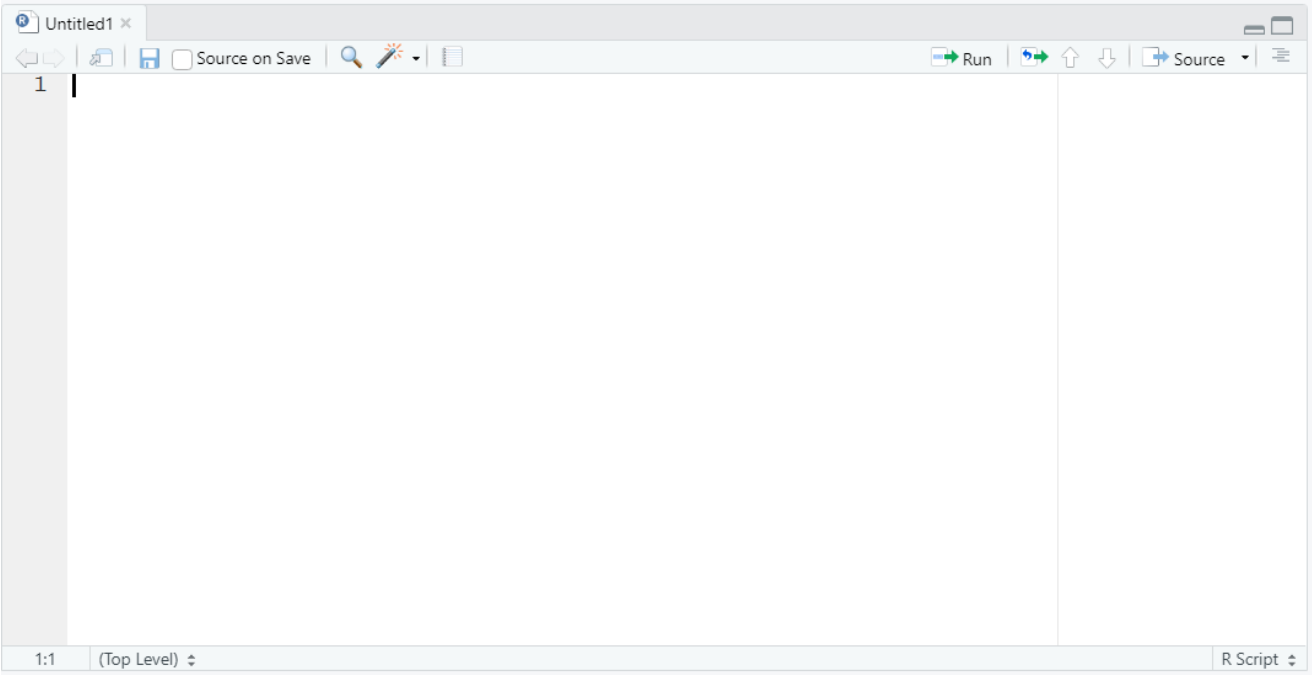
CREAR UN SCRIPT EN RSTUDIO

Un script son archivos que son un conjunto de órdenes que se guardan en un archivo de texto que ocupa muy poco espacio, es decir, es ligero.. El script puede ser ejecutado por lotes o línea línea por un intérprete.

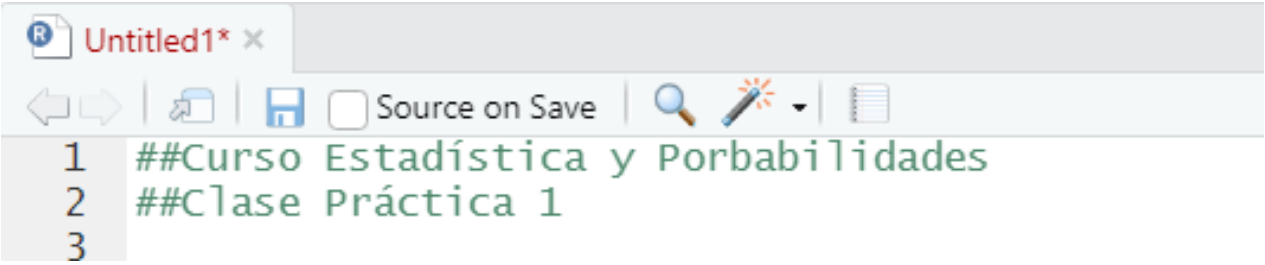
Para crear uno nuevo vamos a la barra File -> New File -> R Script (Ver Figura)



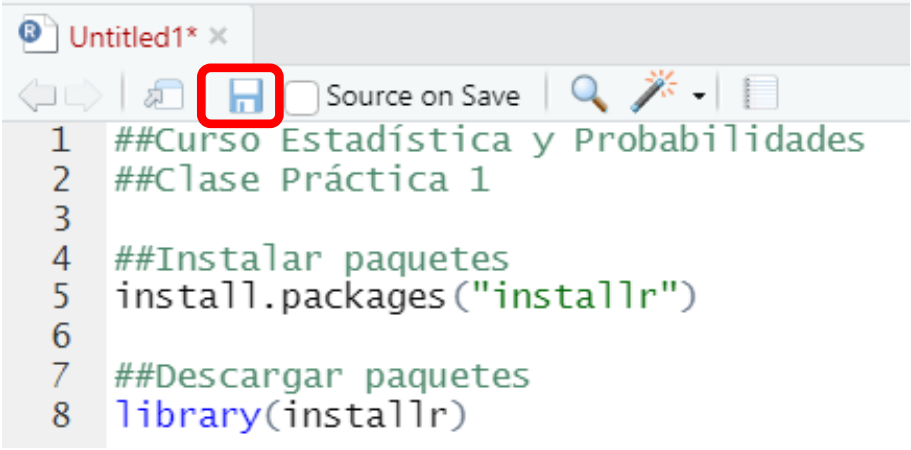
Posteriormente se abrirá la ventana de script en la parte superior izquierda.



En este archivo podemos colocar comentarios anteponiendo el numeral (#), para que identifique que no son comandos, como se muestra en imagen



Entonces podemos ir agregando los comandos que hemos empleado, acompañado de sus indicaciones



Para guardar vamos a ir al icono del diskette como se marca en la imagen anterior. Establecemos un nombre, puede ser Script_Clase1. Ahora veremos que el nombre aparece en la parte superior de la ventana como se muestra en imagen siguiente.

```
Script_Clase1.R
C:/Users/Lenovo/Desktop/UPCH_2022/EyP/Clase_1/Clase_1/Script_Clase1.R
1  ##Curs a y Probabilidades
2  ##Clas
3
4  ##Instalar paquetes
5  install.packages("installr")
6
7  ##Descargar paquete
8  library(installr)
```

LENGUAJE ORIENTADO A OBJETOS

ASIGNACIÓN / CREACIÓN DE OBJETOS

El operador de asignación de valores a un objeto es “=” o “<-“

Los nombres de objetos empiezan por una letra y pueden contener números y símbolos (salvo operadores aritméticos y lógicos). R distingue entre mayúsculas y minúsculas.

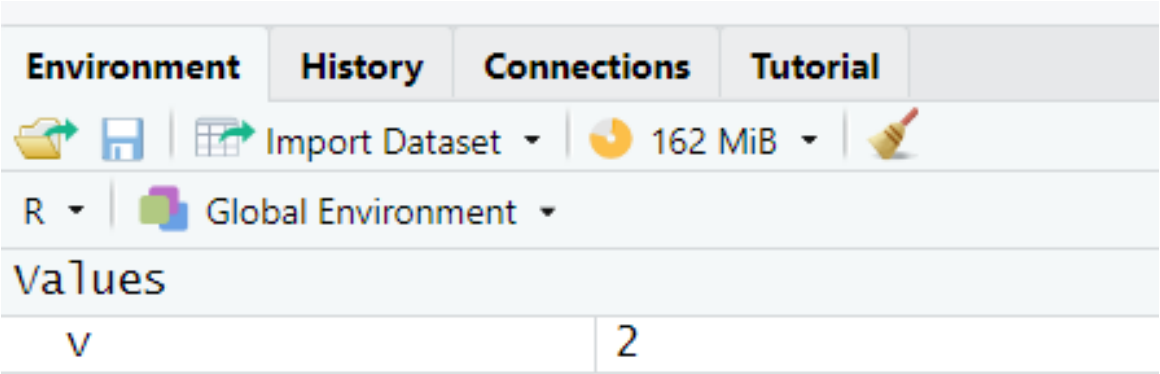
Por lo tanto podemos establecer un objeto denominado “v”, al cual le queremos atribuir el valor 2. Para ello podemos escribir en el script lo siguiente: v=2, marcalos la línea como se ve en la imagen y lo ejecutamos por el Run

```
Source on Save                                   Run
1  ##Curso Estadística y Porbabilidades
2  ##Clase Práctica 1
3
4  ##Instalar paquetes
5  install.packages("installr")
6
7  ##Descargar paquete
8  library(installr)
9
10 v=2
```

Aparece la ejecución en la consola

```
> v=2
>
```

Además, en la ventana de Entorno también observamos el objeto con el valor que se atributo



Para acceder al contenido del objeto, basta escribir el nombre del objeto

```
> v
[1] 2
```

Podemos escribir otro objeto que le asignamos una operación
e<-2+3

```
> e<-2+3
> e
[1] 5
```

Los objetos pueden ser del siguiente tipo:

Integer: Número enteros (…., -1, 0,1,…)
Numeric: Números reales (2.5, 1.7, 3, …)
Logical: TRUE, FALSE
Character: Cadena de caracteres (va entre comillas, por ejemplo “sano”)

Cada objeto tiene unos atributos que determinan sus propiedades:

- Para ver el tipo básico de elementos de un objeto:
  mode(objeto) ***mode(e)***

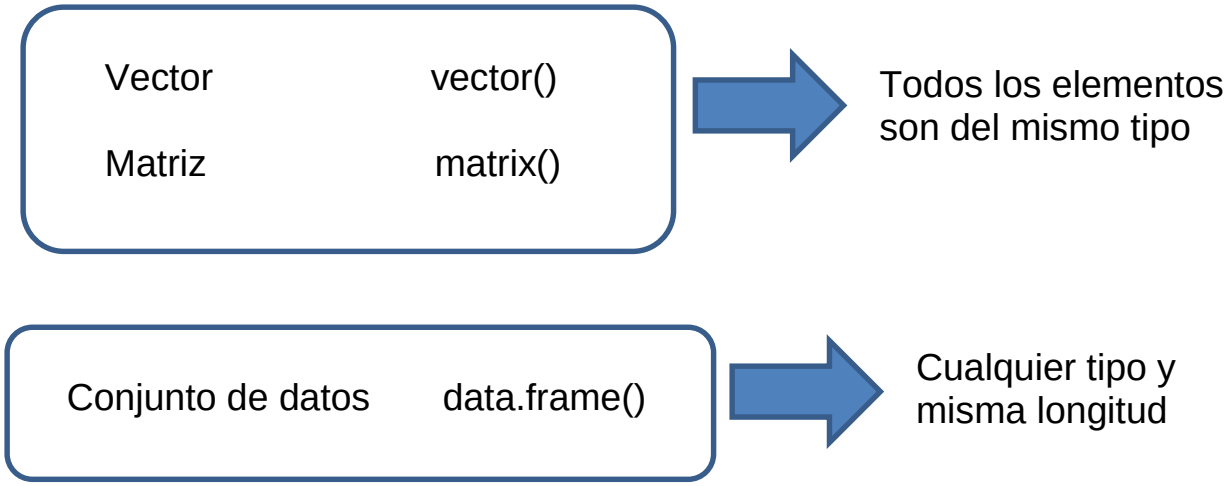
- Para ver el número de elementos de un objeto:
  length(objeto) ***length(e)***

En consola se visualizaría

```
> mode(e)
[1] "numeric"
> length(e)
[1] 1
```

ESTRUCTURA DE DATOS:

Permiten almacenar al mismo tiempo más de un valor



Vectores

Un vector es un espacio de memoria en la computadora que se usa para guardar información de cualquier cosa. Es la estructura de datos más sencilla en R. Un vector es una colección de uno o más datos del mismo tipo.

Para crear los vectores usaremos la función c() que permite combinar uno o más valores.

Entonces podemos tener estos tipos de vectores:

- Vector númerico c(1,2,3,….)
- Vector tipo carácter c(“a”, “b”, “c”, …)
- Vector tipo lógico c(TRUE, FALSE, ….)

Operaciones con vectores

A continuación se presentan los registros de 3 variables (peso, sexo, bajo peso) en 5 recien nacidos

peso	sexo	bpeso*
3200	M	FALSE
2001	F	TRUE
1950	M	TRUE
2650	F	FALSE
2890	M	FALSE

*Se conoce que el recién nacido presenta bajo peso al nacer si es menor de 2500 gr.

Actividad 1:

Vamos a construir un vector para cada variable, pues el vector presenta sus elementos del mismo tipo.

Escribimos en el script lo siguiente, para ejecutarlo:

peso<-c(3200,2001,1950,2650,2890)
sexo<-c("M", "F","M", "F","M")
bpeso<-c(FALSE, TRUE, TRUE, FALSE,FALSE)

Ahora vamos a visualizer nuestros vectores en la ventana de Entorno.

Actividad 2:

Se tiene la información que la balance estuvo descalibrada y estuvo presentando un peso menos 10 gr. Por ello vamos a agregar el valor de 10 gramos a cada registro del peso, pero lo realizaremos en otro vector que vamos a denominar peso_c (es el peso corregido)

peso_c<-peso+10

Actividad 3:

Deseamos verificar que la diferencia de cada registro de peso es de 10 gr
peso_c – peso

En la consola obtenemos los resultados

```
> peso_c - peso
[1] 10 10 10 10 10
```

Actividad 4:

Deseamos saber el tipo de elemento del vector sexo
class(sexo)

En la consola obtenemos los resultados

```
> class(sexo)
[1] "character"
```

Actividad 5

Deseamos conocer el peso corregido promedio del recién nacido

Podriamos emplear la función *sum() y length()*
sum(peso_c)

length(peso_c)
En la consola obtenemos los resultados

```
> sum(peso_c)
[1] 12741
> length(peso_c)
[1] 5
```

Ya tenemos la suma y el numero de observaciones podríamos emplear R como una calculadora

```
> 12741/5
[1] 2548.2
```

Además podemos obtener el valor directamente
sum(peso_c)/length(peso_c)

```
> sum(peso_c)/length(peso_c)
[1] 2548.2
```

Pero también hay una función para el promedio, *mean()*
mean(peso_c)

```
> mean(peso_c)
[1] 2548.2
```

Reconocer obtenemos el resultado en la consola, pero no se ha generado ningún objeto con este valor. Para ello:
resultado=mean(peso_c)

Ahora visualizaras el objeto resultado en el entorno

Seleccionar elementos de un Vector

Podemos estar interesados en seleccionar elementos del vector basados en:

- Posición del elemento: Primera, segunda,….
- Condición: valor mayor que…

Actividad 6

Deseamos identificar el registro (elemento) en la posición 3, de la variable peso_c. Para ello identificamos la posición mediante el corchete, como se muestra.
peso_c[3]

```
> peso_c[3]
[1] 1960
```

Ahora deseamos identificar tanto el registro 3 y 5. Como es mas de un valor empleamos la función *c()*
peso_c[c(3,5)]

```
> peso_c[c(3,5)]
[1] 1960 2900
```

Actividad 7

Deseamos identificar los recién nacidos con bajo peso al nacer considerando el peso del recién nacido corregido

Vamos a generar el objeto bpeso_c (es un nuevo objeto) y establecemos la condición de peso_c menor a 2500 (peso_c<2500)

bpeso_c=peso_c<2500

Actividad 8

Mostrar los pesos corregidos que presentaron bajo peso
peso_c[bpeso_c]

```
> peso_c[bpeso_c]
[1] 2011 1960
```

Actividad 9

Mostrar el género del recién nacido que presentaron bajo peso
sexo[bpeso_c]

```
> sexo[bpeso_c]
[1] "F" "M"
```

Matriz

Las matrices pueden ser descritas como vectores multidimensionales. Al igual que un vector, únicamente pueden contener datos de un sólo tipo, pero además de largo, tienen más dimensiones.

Las matrices son, por lo tanto, una estructura con forma rectangular, con renglones y columnas.

Las matrices se crean con la función *matrix(…)*

matrix(data, ← **CONTENIDO**
nrow =1, ← **FILA**
ncol=1) ← **COLUMNA**

Cambiar nombre con la función *colname* y *rowname*

Actividad 10

Previamente hemos creado vectores, de los cuales tenenos dos vectores numéricos: peso y peso_c, que contienen la información del peso del recién nacido y el peso corregido respectivamente.

Con estos 2 vectores creemos la matriz denominada matriz. Recordar que los registros eran de 5 recien nacidos (5 filas) y tenemos 2 variables (5 columnas)

```
matriz<-matrix(c(peso, peso_c),
          nrow = 5,
          ncol = 2)
matriz
```

En la consola se visualiza asi:

```
> matriz
     [,1] [,2]
[1,] 3200 3210
[2,] 2001 2011
[3,] 1950 1960
[4,] 2650 2660
[5,] 2890 2900
```

Ahora si deseamos obtener el número de dimensiones de una matriz empleamos la función *dim()*

```
dim(matriz)
```

```
> dim(matriz)
[1] 5 2
```

Actividad 11

En la matriz generada, deseamos agregar el nombre de columna y fila, para ello emplearemos la función *colnames()* y *rownames()* respectivamente, después asignamos el vector con el nombre de las columnas, el número de elementos del vector debe coincidir con el número de columnas. De manera similar para las filas

```
colnames(matriz)<- c("peso", "peso_c")

rownames(matriz)<-c("rn1", "rn2", "rn3", "rn4", "rn5")
```

Ahora lo visualizamos asi en la consola

```
> matriz
    peso peso_c
rn1 3200   3210
rn2 2001   2011
rn3 1950   1960
rn4 2650   2660
rn5 2890   2900
```

Actividad 12

Se requiere saber algunas informaciones especificas de la matriz

¿Cuánto es el peso sin corregir del recién nacido número 5? Para identifica el elemento empleamos corchetes, dentro identificamos la fila y la columna

matriz[5,1]

```
> matriz[5,1]
[1] 2890
```

Pero también en lugar de colocar la ubicación podemos emplear los nombres de la fila y columna

matriz["rn5","peso"]

```
> matriz["rn5","peso"]
[1] 2890
```

Actividad 13

Se desea conocer toda la información del recién nacido numero 3

En este caso debo seleccionar una fila (registro rn3). Dentro del corchete colocamos la ubicación de la fila, luego coma seguido de la ubicación de la columna.
Pero en este caso nos interesa seleccionar la fila 3 , entones lo haremos de la siguiente manera, dentro del corchete especificaremos la fila, colocamos la coma y no especificamos la columna, pues requerimos seleccionar una fila en este caso.

matriz[3,]

```
> matriz[3,]
  peso peso_c
  1950   1960
```

Actividad 14

Se desea conocer los pesos corregidos de los cinco recién nacidos

En este caso debo seleccionar una columna (variable peso_c). Dentro del corchete colocamos coma dejando libre la asignación de la fila, pero luego de la coma especificamos la columna de interés.

matriz[,2]

```
> matriz[,2]
 rn1  rn2  rn3  rn4  rn5
3210 2011 1960 2660 2900
```

Data frames

Los data frames son estructuras de datos de dos dimensiones (rectangulares) que pueden contener datos de diferentes tipos, por lo tanto, son heterogéneas. Esta estructura de datos es la más usada para realizar análisis de datos y seguro que resultará familiar si ha trabajado con otros paquetes estadísticos.

Podemos entender a los data frame como una versión más flexible de una matriz.Mientras que en una matriz todas las celdas deben contener datos del mismo tipo, los renglones de un data frame obtienen datos de distintos tipos, pero sus columnas conservan la restricción de contener datos de un solo tipo.

En términos generales, los renglones en un data frame representan casos, individuos u observaciones, mientras que las columnas representan atributos, rasgos o variables.

Previamente hemos creado por 3 vectores que contienen la información del peso corregido (peso_c), el bajo peso al nacer corregido (bpeso_c) y sexo del recién nacido (sexo)

Con estos 3 vectores creemos el data frame denominada ejemplo.

```
ejemplo<-data.frame(peso_c,
                    bpeso_c,
                    sexo)
```

ejemplo

En la consola se visualiza asi:

```
> ejemplo
  peso_c bpeso_c sexo
1   3210   FALSE    M
2   2011    TRUE    F
3   1960    TRUE    M
4   2660   FALSE    F
5   2900   FALSE    M
```

Además puedo solicitar visualizarlo con la función *view()*

View(ejemplo)

Se abrirá otra pestaña en la ventana script

Script_Clase1.R × ejemplo ×

Filter

	peso_c	bpeso_c	sexo
1	3210	FALSE	M
2	2011	TRUE	F
3	1960	TRUE	M
4	2660	FALSE	F
5	2900	FALSE	M

Además lo podemos visualizar desde la ventana del entorno, dando click donde se señala

Environment History Connections Tutorial

Import Dataset 142 MiB List

R Global Environment

Data

ejemplo 5 obs. of 3 variables

Podemos obtener información del data frame mediante las siguientes funciones

dim()
dim(ejemplo)

```
> dim(ejemplo)
[1] 5 3
```

Me indica son 5 filas (registros de recién nacidos) y 3 columnas(variables o vectores)

length()
length(ejemplo)

```
> length(ejemplo)
[1] 3
```

En el caso de data frame length() nos refiere el número de columnas

names()
names(ejemplo)

```
> names(ejemplo)
[1] "peso_c"  "bpeso_c" "sexo"
```

Actividad 15

Se requiere saber algunas informaciones especificas del data frame.

¿Cuánto es el peso del recién nacido número 5? Para identifica el elemento empleamos corchetes

ejemplo[5,1]

```
> ejemplo[5,1]
[1] "2900"
```

Pero también en lugar de colocar la ubicación podemos emplear los nombres de columna

ejemplo[5,"peso_c"]

```
> ejemplo[5,"peso_c"]
[1] 2900
```

Ahora ¿Cuánto es el peso del recién nacido 3? y ¿cuál es el genero del recién nacido 5?

ejemplo[c(3,5), c(1,3)]

```
> ejemplo[c(3,5), c(1,3)]
  peso_c sexo
3   1960    M
5   2900    M
```

ejemplo[c(3,5), c("peso_c","sexo")]

```
> ejemplo[c(3,5), c("peso_c","sexo")]
  peso_c sexo
3   1960    M
5   2900    M
```

Actividad 16

Se desea conocer toda la información del recién nacido numero 3

En este caso debo seleccionar una fila . Dentro del corchete colocamos la ubicación de la fila, luego coma seguido de la ubicación de la columna.
Pero en este caso nos interesa seleccionar la fila 3 , entones lo haremos de la siguiente manera, dentro del corchete especificaremos la fila, colocamos la coma y no especificamos la columna, pues requerimos seleccionar una fila en este caso.

ejemplo[3,]

```
> ejemplo[3,]
 peso_c bpeso_c    sexo
 "1960"  "TRUE"     "M"
```

Actividad 17

Se desea conocer los pesos de los cinco recién nacidos

En este caso debo seleccionar una columna (variable peso_c). Dentro del corchete colocamos coma dejando libre la asignación de la fila, pero luego de la coma especificamos la columna de interés.

ejemplo[,"peso_c"]

```
> ejemplo[,"peso_c"]
   rn1    rn2    rn3    rn4    rn5
"3210" "2011" "1960" "2660" "2900"
```

EJERCICIO PROPUESTO

Una compañía farmacéutica conduce un estudio piloto para evaluar tres dosis de un nuevo agente hipnótico. Para cada participante del estudio se registro el tiempo de sueño y el género

Tiempo (horas)	Dosis (mm/kg)	Sexo
4	3	F
6	3	M
5	3	F
9	10	M
8	10	F
7	10	M
13	15	F
11	15	M
9	15	F

1.- Generar un vector para cada variable
2.- Generar una matriz para los vectores numéricos
3.- Identificar en la matriz generada el participante número 5 que tiempo de sueño presento y dosis del fármaco.
4.- Identificar en la matriz generada el participante número 1 que tiempo de sueño presento y en el participante número 9 que dosis del fármaco empleo.
5.- Genera un data frame con los tres vectores generados. Indicar el número y nombre de columnas
6.- Identificar en el data frame, el género del participante 7.
7.- Identificar en el data frame, el tiempo del particpante 2 y la dosis del participante 8.